AGRITECH

Implementation, Architecture, Database, Authentication, Feature Workflows, Interview and Presentation Guide

Prepared from the repository source and project README. This guide explains what the code is designed to do, how a request moves through it, and which implementation details are useful to discuss in a technical interview. It separates registered routes from standalone modules and flags configuration or readiness dependencies.

Project team	Roll number
ARYAMAN SANSKRITYAYAN	23051090
ANKIT MUKHERJEE	23051084
ANSHIKA NATH	2305765
DEV KUMAR	2305292

Program: CSE **Guide:** Ms. Chandani Kumari **Academic year:** 2026–2027

Important: This is a code-informed explanation, not a claim that every feature is deployed, integrated, or validated. Runtime behavior depends on installed dependencies, environment variables, database state, and model artifacts.

1. Project in one minute

AgriTech is a multi-module web application for agricultural decision support and related farmer services. It combines browser pages with a Python Flask server, selected machine learning services, SQLAlchemy persistence, and optional integrations. The project aims to make crop guidance, yield estimates, disease screening, farm information, market services, and community tools available from one platform. A clear interview description should call it an integrated prototype/platform and qualify which services depend on configuration or trained model files.

Purpose and users

• Farmers: explore crop, fertilizer, yield, weather, irrigation, and disease information.

• Buyers, shopkeepers, experts, and administrators: use role-specific marketplace, community, or management experiences where enabled.

• Project demonstrators: show the architecture and feature journeys through the web interface.

Problem addressed

Agriculture workflows are fragmented across separate sources. The platform offers a common entry point for data-driven guidance and services. Prediction outputs support decisions; they do not replace local agronomists, laboratory soil tests, or field verification.

What makes the system technically interesting

• A Flask monolith coordinates HTML pages, route handlers, services, and model calls.

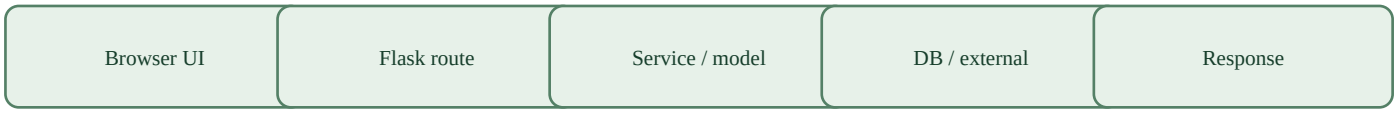
• Feature modules include synchronous form flows and asynchronous Celery tasks.

• SQLAlchemy models cover users and domain records; migrations support schema evolution.

• JWT auth and role checks are available in API code; feature routes should be checked individually for protection.

• Some integrations and feature blueprints are optional or commented out, so route presence in the repository is not proof of production availability.

2. Architecture and request lifecycle



Typical lifecycle: a page script or HTML form sends a GET/POST request; Flask matches a route or registered blueprint; validation and sanitization run; the handler calls a domain service, model, or database; a response is rendered as HTML or JSON; the browser updates the page. Long-running tasks may be enqueued in Celery and polled through a task-status route.

Main components

Layer	Implementation seen in repository	Purpose
Frontend	HTML templates/pages, CSS, JavaScript	Forms, dashboards, navigation, validation, fetch/AJAX interactions
Application	Flask app.py and registered Blueprints	Routing, configuration, error handling, service coordination
Domain/services	backend/services, agri_utils.py, feature packages	Agronomic rules, ML inference, analytics, business logic

Layer	Implementation seen in repository	Purpose
Persistence	Flask-SQLAlchemy models, migrations	Structured records, users, transactions, audit and feature state
Async / realtime	Celery, Redis configuration, Flask-SocketIO event handlers	Background work and event delivery where configured
Integrations	Firebase, Gemini, SMTP and external data services	Optional authentication/config, AI, mail and data capabilities

Bootstrap and configuration

app.py loads environment values, creates the Flask app, reads the selected config, sets upload storage, initializes db, migrations, mail and rate limiting, configures Celery and audit middleware, imports User, configures CORS, then registers Blueprints. The README identifies app.py as the primary entry point. Secrets belong in .env or deployment secrets, not source control. A development SECRET_KEY fallback exists in settings; deployments must override development defaults.

3. Frontend: pages and interaction model

The UI is built primarily from HTML, CSS and browser JavaScript, with many standalone pages at the repository root and feature directories. A page typically collects form values, validates obvious input client-side, submits a request, and presents the server result. Client validation improves usability; the server must still validate because clients can be bypassed.

Feature area	Representative page/code family	What to explain in presentation
Core navigation	index.html, main.html, navbar.html	Entry point and navigation to agricultural/service modules
Accounts	login.html, register.html, auth.js	Registration/login UI and token handling; backend auth is /api/auth/*
Crop advice	crop_recommendation/*, crop.html	Soil/climate input to crop prediction/advice
Disease	disease.html, ai_disease.html, disease.js	Upload/capture image, send it for classification or advisory
Yield	yield_prediction.html via Flask /yield/	Form for crop/location/agronomic values and prediction response
Weather/farm	weather*, farm_dashboard*, climate_dashboard*	Current conditions, farm context and advisory display
Irrigation/rotation	irrigation*, crop_rotation.html	Scheduling/recommendations based on field conditions
Marketplace/community	marketplace.html, farmer.html, community_forum.html	Farmer interaction and commerce/service discovery
Prices/finance	Crop_Prices_Tracker and loan pages	Market trend/forecast or financing-related workflows; external/model dependencies apply

Practical front-end interview points: distinguish static page rendering from API calls; explain how loading/error/success states are handled in the relevant JS; state whether tokens are sent in Authorization headers or cookies by inspecting that page’s code; and show one actual browser journey. Do not claim all UI pages use a shared SPA framework: the project is largely multi-page HTML/JS.

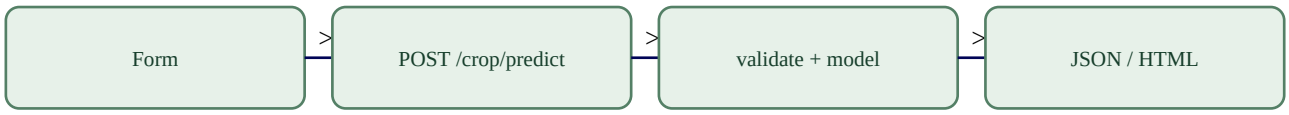
4. Backend routes and API surface

The primary route inventory below comes from explicit decorators and Blueprint registration in app.py. Blueprints add prefixes during registration; the listed endpoint is the effective public path where derivable. A route may still depend on configuration, request data, DB availability, or model files.

Purpose	Effective route(s)	Methods / notes
Pages	/ , /farmer, /shopkeeper, /main, /about, /blog, /contact, /chat	GET render routes
Crop model	/crop/, /crop/predict, /crop/download_report	crop_bp registered with /crop prefix; GET/POST prediction/report
Async crop	/api/crop/predict-async, /api/task/	POST enqueue; GET status
Fertilizer	/api/recommend/fertilizer	POST; request validated and delegated to fertilizer logic
Yield	/yield/, /yield/predict	GET form; POST prediction; service is loaded at startup and may be unavailable
Authentication	/api/auth/register, /login, /refresh, /logout, /me, /validate	POST except GET /me and /validate; JWT flow
Files	/upload, /, /download/, /user/	files_bp route paths; file authorization/validation must be checked in handler
Ingestion	/api/v1/ingest/upload, /api/v1/ingest/status/	POST upload; GET ingestion tracking status
Spatial analytics	/fields, /fields, /analyze/	spatial_bp paths; create/list/analyze field records
Irrigation / rotation	/api/irrigation-schedule, /rotation, /api/analyze-rotation	irrigation and rotation Blueprints
Prices	/crop_price_tracker, /get_states, /get_markets, /api/forecast, /api/optimize-profit, /api/heatmap, /api/alert-subscribe	tracker_bp route family; combinations GET/POST
Loans / reports	/process-loan, /generate-loan-report, /download-report/, /task-status/	sync/async paths; reports and task polling
Health / model docs	health Blueprint, model-versioning Blueprint, Swagger Blueprint	Additional routes are defined in the respective modules

Other route families exist in separately runnable sub-apps. For example crop_recommendation/app.py defines a standalone Flask app, but the main app imports crop_recommendation.routes. When asked “what is the endpoint?”, state whether you mean the main server or a standalone module.

Representative request flow: crop recommendation



The crop Blueprint is registered at /crop. The form supplies feature values; the route checks input, converts expected fields, invokes the configured recommendation/model path, and returns a prediction or an error/validation response. The asynchronous route /api/crop/predict-async instead submits a Celery task and returns a task identifier; the client checks /api/task/ until completion. This division protects request latency for work that can take longer.

Representative request flow: yield prediction

GET /yield/ renders a prediction page. At app startup, app.py attempts to construct YieldPredictionService once; on failure it stores an error and disables prediction handling gracefully. POST /yield/predict parses submitted values,

calls service.predict, and re-renders with result. Value errors produce a client error; unexpected errors are logged and presented as a generic prediction failure. Explain that this is server-rendered HTML rather than necessarily a JSON API.

5. Feature-by-feature implementation and purpose

5.1 Crop recommendation

Purpose: rank or predict crops likely to suit soil/climate inputs, helping farmers narrow choices. The repository has crop recommendation packages, training scripts, model assets and an integrated Blueprint. A supervised model learns an association between input features and crop labels. The route layer receives values; model/service logic transforms features into the model's expected order and returns a label or recommendation. Discuss feature units, missing-value handling, training/validation split, model choice and region bias only after checking the exact model file and current validation results. The README mentions Random Forest/XGBoost as possible methods, but the active endpoint implementation should be your authority.

5.2 Fertilizer recommendation

Purpose: suggest nutrient inputs based on soil pH, crop and growth stage. app.py exposes POST /api/recommend/fertilizer and imports recommend_fertilizer from agri_utils.py. The utility layer combines crop guidance/rule logic and may use a loaded predictive component. Explain inputs and outputs from actual code; recommendations must be framed as advisory and calibrated to local soil-test data. Never invent an N-P-K dose or claim agronomic validation without a reference.

5.3 Crop yield prediction

Purpose: estimate yield from structured agronomic inputs. backend/services/ml/yield_prediction.py loads the trained model and encoders/scalers once. predict() validates/encodes request fields, applies the same transformations as training, invokes model.predict, checks the returned value is finite, and formats a nonnegative rounded yield result. Key interview concept: preprocessing at inference must match training. Mention data source, target unit, metrics, and test split only after verifying the training notebook/model evaluation.

5.4 Plant disease diagnosis

Purpose: screen leaf/crop images for disease-like visual classes. backend/services/ml/plant_disease_classifier.py defines inference wrapper and architecture for a CNN_NeuralNet checkpoint under Disease prediction; it expects checkpoint and class metadata. Missing artifacts cause a clear unavailable error. Separate AI advisory service may explain prevention/treatment, but image classification and language-model advice are distinct steps. The main app.py shows disease Blueprint import commented out, so do not present that Blueprint as active on the primary Flask server unless another route is verified. Some processing task paths can return simulation/fallback output; disclose this in demos.

5.5 Weather and advisory

Purpose: turn weather conditions into operational reminders such as irrigation timing or disease risk. agri_utils.py includes rule-based advisory and disease risk calculations based on temperature, humidity, and rainfall; weather services and pages exist. A rule-based risk heuristic is not the same as a trained disease forecasting model. Explain thresholds and source freshness only from code/config.

5.6 Irrigation and crop rotation

Purpose: irrigation scheduling helps align watering with crop needs and conditions; rotation advice encourages better crop sequencing and soil/pest management. The main app registers irrigation_bp and rotation_bp. Present these as separate endpoints and domain modules: schedule generation and rotation analysis are not automatically connected to crop model output unless the code explicitly passes those values through.

5.7 Yield mapping / spatial analytics

Purpose: represent field boundaries/locations and analyze geospatial patterns. spatial_bp is registered; routes create/list fields and analyze a field. Related models/services support disease incidents, yield grids, and spatial calculations. This area involves geospatial data representation and query logic. Confirm coordinate reference system, validation, and actual map visualization before claiming accuracy.

5.8 Crop prices tracker

Purpose: help users inspect market locations/prices, forecasts, profit optimization, heatmaps and alerts. Crop_Prices_Tracker/app.py is imported as tracker_bp. Routes provide state/market lookup, forecast, profit optimization, heatmap and alert subscription. Forecast quality depends on historical data and model assumptions; explain source, freshness, horizon and baseline when known.

5.9 Marketplace, community and farmer services

Purpose: connect farmers and service providers around produce, supplies, advice, labor, equipment and knowledge exchange. Many HTML pages and backend models/services exist. The repository's breadth includes marketplace, equipment, procurement, warehouse, schemes, insurance, finance, and community features. Explain a feature as implemented only if you can show its actual route, DB write/read, and working UI flow. Some pages may be prototypes or mock data.

5.10 Chatbot and AI integrations

Purpose: answer agricultural questions and guide users through platform features. The repository includes chatbot front ends, response data, and a Gemini integration dependency. External AI features require valid credentials and network access. Explain the prompt/input flow, provider, safety behavior and fallback from the exact deployed configuration. A static response script is distinct from a live generative model.

A simple explanation before the technical details

Imagine AgriTech as a service desk. The web page is the counter where a farmer asks for help. A route is the sign that sends each request to the correct desk. The service code is the staff member doing the work. A machine learning model is a trained specialist that has learned patterns from examples. The database is the filing cabinet that keeps records. The response is the answer handed back to the farmer. This picture helps explain the system before learning any programming terms.

Technical word	Plain meaning	AgriTech example
Frontend	What the user sees and touches	A form to enter soil values
Backend	The hidden program that handles requests	Flask receives a recommendation request
Route	A named web address and action	POST /yield/predict asks for a yield estimate
Database	Organized long-term records	User and farm records stored as rows
Model	A program trained on examples to make estimates	Crop or yield prediction artifact
Authentication	Checking who is signed in	Login checks password and issues token
Authorization	Checking what a signed-in person may do	Role check allows only permitted actions
API	A precise message format between programs	Browser sends JSON and receives JSON
Queue / worker	A waiting line and a separate worker for long jobs	Celery task runs after web request returns

One full request, slowly explained

1. A person opens a page in the browser. HTML gives it structure, CSS makes it readable, and JavaScript can react to clicks.

2. The person enters information and presses a button. The browser gathers those values.

3. The browser sends an HTTP request to the AgriTech server. GET usually asks for a page or information; POST usually sends data or asks the server to perform an action.

4. Flask compares the web address and method with registered routes. A Blueprint is a folder-like group of related routes.

5. The route checks that the message has the required fields and sensible values. Never trust only browser checks because a person can send a request without the normal page.

6. The route calls the appropriate service. That service may apply rules, call a trained model, read or write the database, or contact an optional outside service.

7. Flask turns the result into a reply, usually a web page or JSON data. Errors should be understandable to the user and details should go to protected logs.

8. The browser shows the reply. A long task can instead return a task ID so the browser can ask for progress later.

Feature stories in everyday language

Farmer account and profile

Purpose: remember who the user is and which role they have. The person submits account details; the server validates them, stores a password hash rather than the original password, and creates a user row. At login, the server checks the entered password against that hash and returns tokens. Protected requests include proof of identity. A role check then decides whether that identity may perform the requested action.

Crop recommendation story

A farmer opens the crop page, enters soil nutrients N, P, K, acidity (pH), temperature, humidity, and rainfall. The browser sends these values to the crop prediction route. The route checks that values exist and fit expected ranges. The model returns a crop class; a label encoder turns a numeric class back into a crop name when needed. The server renders the result page. If a model is missing, the person sees an unavailable message instead of a fake answer.

Fertilizer recommendation story

A person provides soil and crop context. The server validates it, then agri_utils recommendation logic uses the available crop rules and inputs to form guidance. The response is advice for consideration, not a guarantee. Fertilizer rates must be based on local soil testing and authoritative agronomy guidance; a software output alone cannot prove the correct field dose.

Yield prediction story

The user selects a crop, season, state, year, area, rainfall, fertilizer, and pesticide values. The yield service checks that the selected categories exist in the encoders and the numbers are valid. It arranges values in the model's expected order, uses the saved scaler and encoders from the training bundle, runs prediction, and returns an estimate with the data unit caveat. Consistent training and prediction preprocessing is essential.

Plant disease image story

The user supplies a crop image. An image classifier can convert pixels into a predicted class. Separate advisory logic may then explain possible next steps. These are two different jobs: recognizing a visual pattern and generating or retrieving advice. Image lighting, leaf angle, background and disease variety affect the prediction. In the main app, the disease Blueprint registration is commented out, so check an active endpoint before claiming this is served by the main application.

Weather, climate and alerts story

Weather pages and climate services display environmental information. Rule-based functions can combine inputs such as temperature, humidity and rainfall to estimate risk or suggest an action. Such a rule is only as current and accurate as the data source and threshold assumptions. A risk alert is not the same thing as confirming a disease.

Irrigation and crop rotation story

Irrigation tools help schedule or manage water delivery; rotation tools examine crop sequences and suggest alternatives. Their pages and registered route modules represent distinct workflows. Do not assume a crop recommendation automatically configures irrigation: values must be connected in code for that handoff to happen.

Market prices and marketplace story

The price tracker can retrieve or present market choices, forecasts, optimization, heatmaps and subscriptions through its route family. Marketplace pages support buying, selling or service discovery where their actual route and data path are implemented. A forecast depends on its data source and time period. A marketplace screen backed by sample data is a prototype until a real persistent transaction flow is demonstrated.

Community, schemes and service modules

The repository contains modules for community discussions, government schemes, labor, equipment, procurement, warehouse, insurance, finance, processing, logistics, sustainability, carbon and ledger activity. Their purpose is to support more than crop prediction: connecting people, services, goods and records. Each module should be explained by tracing its own page to its registered route, service, database models, and response. A model class by itself does not prove the complete user journey is active.

Files and data ingestion story

A file upload begins in a browser form. The server should check file size and type, create a safe storage name, associate metadata with an owner, and return a tracking or file identifier. Ingestion may process the file in stages and provide a status endpoint. A download must verify that the requesting user is allowed to access that file. Exact checks differ by endpoint and should be confirmed in the handler.

Chatbot and external AI story

The chat page collects a question and sends it to client-side response logic or an AI provider, depending on the feature path. A live Gemini integration requires a valid key and network access. Static canned responses do not call a generative model. Explain which path is active in your demo and how unsupported or unsafe questions are handled.

Loans and generated reports story

A loan form sends structured information to a synchronous or asynchronous processing route. Validation checks the request; business logic creates a result or schedules a worker. A report endpoint can generate a downloadable file. The download route must restrict paths and access. Financial predictions or eligibility outcomes should be presented as software output and not as guaranteed lending decisions.

Reading the route list correctly

A route is identified by both its path and HTTP method. For example, GET /yield/ displays the form, while POST /yield/predict sends values for processing. The same path can support more than one method. Blueprint registration adds a prefix to the route written inside a feature module. The main application also has standalone pages and compatibility routes. This repository is large; its public routes are spread across many files, so the route table summarizes the core journeys rather than pretending every page is one API. For a particular screen, search its JavaScript for fetch or form action, then follow that URL to the registered handler.

6. Database and data model

Flask-SQLAlchemy provides the ORM. app.py initializes db and Flask-Migrate; backend/models contains domain models. Configuration defaults to SQLite agritech_dev.db in development settings, while deployment can supply DATABASE_URL. Therefore do not claim the production database is SQLite unless the environment confirms it. Migrations describe schema changes; model classes define entities and relationships.

Model family	Example models	Why it exists
Identity	User, Token, UserSession	Account identity, password hash, token/session metadata

Model family	Example models	Why it exists
User activity	Notification, File, AuditLog	Notifications, uploaded file metadata, audit trail
Prediction	PredictionHistory, CropDiagnosticReport	Persist prediction or diagnostic records where invoked
Agronomy	SoilTest, FertilizerRecommendation, IrrigationZone, IrrigationSchedule	Soil observations and crop input/scheduling records
Weather / climate	WeatherData, CropAdvisory, ClimateZone, TelemetryLog	Conditions, derived advisories, sensor observations
Market / trade	MarketPrice, MarketplaceListing, MarketplaceOrder, SupplyBatch	Price tracking and commerce/traceability
Farm / equipment	Farm, FarmMember, FarmAsset, Equipment, RentalBooking	Farm organization and machinery workflows
Risk / services	InsurancePolicy, ClaimRequest, GovernmentScheme, Loan records	Financial, public program and risk workflows

How to explain DB access

A route should not embed raw SQL throughout the UI layer. It calls model queries through SQLAlchemy, creates or updates model objects, adds them to db.session, commits a transaction, and serializes a safe response. On exceptions, rollback avoids leaving a failed transaction open. Unique constraints/indexes protect identifiers such as User.username and email. Relationships express connected data; lazy loading and serialization need care to avoid excess queries or leaking private fields.

Privacy/security talking point: file metadata and user records may be stored in DB while binary uploads live in an upload/storage directory. Verify allowed file types, size limits, ownership checks and safe download paths in the specific endpoint before claiming secure file handling.

7. Authentication, authorization and security

Registration

POST /api/auth/register requires username, email, password, full_name and role. It validates required data, sanitizes strings, checks username/email/name/password/role, rejects duplicates with 409, creates User, hashes the password with Werkzeug, commits, then returns a user profile. Registration currently validates email using gmail_only=True; mention this constraint accurately if discussing current behavior.

Login and tokens

POST /api/auth/login accepts username or email plus password, verifies the stored password hash, and issues JWT tokens. The route documentation says access token is returned and refresh token is placed in an HTTPOnly cookie. Refresh, logout, /me and /validate routes support lifecycle and identity checks. Confirm token storage and Authorization header behavior in the specific frontend before describing client handling.

Authorization

Authentication proves identity; authorization decides permissions. JWT decorators reject missing/invalid/expired credentials. Role-based checks can restrict actions to roles such as farmer, shopkeeper, admin, consultant or other allowed roles. A route must apply the appropriate decorator consistently; the existence of roles in User does not mean every endpoint is protected.

Security measures visible in code/config

• Werkzeug password hashing instead of storing plaintext passwords.

• Input validation and sanitization utilities; SQLAlchemy parameterized ORM queries.

• CORS configured from allowlisted origins.

• Flask-Limiter, audit middleware, secret/environment configuration.

• File endpoints and tokens must be reviewed for ownership, expiration, CSRF/cookie behavior, upload validation and secret rotation.

Good critique: development fallback secrets and permissive local origins are convenience defaults; production must use strong unique secrets, HTTPS, restrictive CORS, secure cookie attributes, key rotation, and hardened debug settings. Do not print .env contents or credentials in a demo.

8. Asynchronous tasks and realtime events

Celery offloads work such as crop prediction, loan processing, report synthesis or other background jobs. A web request validates and enqueues a task; broker/result backend (commonly Redis according to requirements/config) transports work; worker executes it; client polls a task-status endpoint or receives Socket.IO events when handlers are wired. Async is useful for latency-heavy or multi-step processing. It adds operational needs: broker/worker deployment, retries, idempotency, timeouts, result expiry and monitoring. A configured task module does not guarantee a worker is running.

9. Testing, evaluation and quality

The repository includes pytest configuration and security-related tests. Use layered testing: unit tests for utility/model transformations; route tests for status codes and schemas; integration tests for DB and model artifacts; end-to-end browser checks for complete journeys. For ML, report a held-out evaluation set, class distribution, accuracy/precision/recall/F1 or regression MAE/RMSE/R^2 as appropriate. Distinguish software correctness from model quality and agronomic utility.

Test	Input	Expected assertion
Crop API	Valid numeric soil/climate payload	Successful response with documented crop result shape
Crop validation	Missing, malformed, or out-of-range values	4xx response; no traceback leaked to user
Yield	Valid form values and model artifact	Finite prediction and correctly labeled unit
Yield unavailable	Missing model artifact	Graceful unavailable message, app remains diagnosable
Auth register	New valid account	201; password stored only as hash
Auth duplicate	Existing username/email	409; no duplicate row
Auth protected route	No/invalid/expired token	401/403 according to decorator contract
File upload	Disallowed type/oversized file/other user id	Rejected and cannot access another user’s file
Async task	Submit and poll queued task	Known task ID; transitions pending to success/failure

No measured test metrics, performance numbers, user study results, or deployment scale are supplied here; do not invent these in a presentation.

10. Presentation flow (7–10 minutes)

Time	Slide / talking point
0:00–0:45	Problem and objective: fragmented agricultural guidance; AgriTech unifies selected tools.
0:45–1:30	Users and scope: farmers, crop advice, yield, disease, irrigation, market/community modules.
1:30–2:30	Architecture: browser → Flask → services/models → DB/external integrations → response.
2:30–4:30	Live demo: choose one complete journey, e.g. crop recommendation or yield form; show input, response, validation.
4:30–5:30	Explain one ML pipeline: input features, preprocessing, inference, output, limitations.
5:30–6:30	Database and authentication: SQLAlchemy, User, hashed password, JWT and role checks.
6:30–7:30	Testing/evaluation: tests actually run; ML metrics only if documented.
7:30–8:30	Limitations and future work: data validation, field trials, offline support, security hardening.
8:30–9:00	Conclude with impact goal and invite questions.

Demo checklist

• Run the app and any required worker before presenting; verify models and sample data load.

• Use synthetic/non-sensitive demo accounts and inputs.

• Check network-dependent services in advance; prepare a fallback screenshot/recording.

• Show one feature end-to-end instead of clicking many disconnected pages.

• Be ready to explain what happens when input is invalid or a model file is absent.

• Never expose API keys, personal data, or .env values on screen.

11. Interview questions with answer anchors

Why Flask? A lightweight Python web framework fits a modular server-rendered/API application and integrates with Python ML libraries. Blueprints organize route families.

What does a Blueprint do? It groups routes and related handlers so features can be registered with a URL prefix; main app registers multiple Blueprints.

How does a prediction request work? Validate payload → normalize/encode features → call model/service → format response → return JSON or render template; async route enqueues Celery.

Why preprocess features? Models expect the same feature order, scaling and encodings used during training. Mismatch silently degrades predictions.

How are passwords secured? Werkzeug generates a password hash; login checks candidate password against hash. Plaintext is not stored.

JWT vs session? JWT is a signed claim token sent with requests; server verifies signature/expiry. Cookie refresh token adds browser lifecycle considerations. Do not conflate authentication with role authorization.

What is ORM/migration? SQLAlchemy maps Python classes to relational tables and manages queries; Flask-Migrate/Alembic tracks schema revisions.

Why async task queue? Move long work out of request-response latency; task ID lets clients poll or receive events. Requires broker and worker operations.

How measure crop classifier? Use leakage-free held-out data; report accuracy and per-class precision/recall/F1, confusion matrix, class balance and regional limitations.

How measure yield regression? MAE/RMSE/R^2 on held-out data, with target units and baseline; inspect errors by crop/region/season.

What if model is missing? Startup/service should expose readiness failure and return graceful unavailable error; log detailed diagnostics without exposing secrets.

How protect file uploads? Validate size/type/content, generate safe names, isolate storage, enforce owner authorization, avoid path traversal and scan risky content.

What are the limitations? Data representativeness, external integration availability, model confidence/generalization, varied readiness across modules, and need for field evaluation.

What would you improve first? A prioritized answer: reproducible model/data evaluation, consistent auth/authorization coverage, integration tests, API contracts/observability, and low-bandwidth usability.

12. Repository-grounded implementation notes and caveats

• Main entry is app.py; app registers crop, health, files, spatial, ingestion, model-versioning, irrigation, rotation, authentication, Swagger and price-tracker Blueprints.

• Disease Blueprint registration is commented out in app.py. Discuss disease as an available model/module/UI only with this caveat unless you verify an active route elsewhere.

• README names Random Forest/XGBoost and regression/CNN families at a high level. For exact architecture and metrics, inspect the selected artifact/training pipeline.

• A root catch-all static route and legacy aliases exist; distinguish compatibility routes from primary APIs.

• The repository contains many ambitious domain modules. A model class or page is not proof of a complete deployed feature. Demonstrate real persisted request/response path.

• External integrations (Firebase, Gemini, SMTP, weather/market data) may be disabled without credentials.

• The development database URL defaults to SQLite; config/environment can change it. State the active environment when presenting.

Glossary

Term	Meaning
API	Contract for software components to exchange requests/responses
Blueprint	Flask mechanism for grouping and registering route modules
CNN	Convolutional neural network commonly used for image classification
JWT	Signed token carrying claims, usually with expiry
ORM	Object-relational mapping between Python classes and relational tables
RBAC	Role-based access control
Celery	Distributed task queue for background jobs
Inference	Using a trained model to produce an output for new input
Migration	Versioned database schema change

How to prepare your final defense

Before presenting, run the exact flows you will show, note the commit/environment, identify model file names and data sources, verify active routes and authentication protection, and capture genuine screenshots. Prepare concise evidence for each claim: route, service/model, table if persisted, test, and limitation. If a feature is a prototype, say

so directly; accurate scope control is stronger than unsupported claims.